EXPERIMENT 3 - CLI ,GUI AND VUI

AIM:

To develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) implementations of a Simple Adder program in Python.

TOOLS REQUIRED:

• Python 3.x
• VS Code
• Tkinter Library
• SpeechRecognition Library
• PyAudio Library

1. CLI IMPLEMENTATION (Command Line Interface):

THEORY:

A Command Line Interface (CLI) allows users to interact with a program by typing commands in a text-based environment.

CODE:

```
from file_operations import create_file, rename_file



while True:

    print("\n===== CLI FILE MANAGER =====")

    print("1. Create File")

    print("2. Rename File")

    print("3. Exit")



    choice = input("Enter choice: ")
```

```
    if choice == "1":
        name = input("Enter file name: ")
        print(create_file(name))

    elif choice == "2":
        old = input("Enter old file name: ")
        new = input("Enter new file name: ")
        print(rename_file(old, new))

    elif choice == "3":
        break

    else:
        print("Invalid choice!")
```

HOW IT WORKS:

1. The program imports `create_file` and `rename_file` functions from the file_operations module.
2. A menu is displayed with options to create a file, rename a file, or exit.
3. The user enters a choice based on the required operation
4. If the user selects "Create File", the program asks for a file name and calls the `create_file()` function.
5. If the user selects "Rename File", it asks for the old and new file names and calls the `rename_file()` function.
6. The respective function performs the operation and returns a message which is displayed on the screen.

7. The loop continues to run until the user selects the exit option.
8. If an invalid choice is entered, an error message is displayed.

OUTPUT:

```
PS C:\Users\SHANTHOSH.S\OneDrive\Desktop\UID_EXP3> python cli.py

 ===== CLI FILE MANAGER =====
 1. Create File
 2. Rename File
 3. Exit
 Enter choice: 1
 Enter file name: SHANTHOSH
 File created successfully!

 ===== CLI FILE MANAGER =====
 1. Create File
 2. Rename File
 3. Exit
 Enter choice: 3
PS C:\Users\SHANTHOSH.S\OneDrive\Desktop\UID_EXP3>
```

2.GUI IMPLEMENTATION(GRAPHICAL USER INTERFACE):

THEORY:

A Graphical User Interface (GUI) allows users to interact with a system using graphical components such as buttons, text fields, labels, and list boxes instead of typing commands.

CODE:

```
import tkinter as tk

from tkinter import messagebox

import os

from file_operations import create_file, rename_file



# ---------- FUNCTIONS ----------
```

```python
def create():
    name = file_entry.get()
    if name == "":
        messagebox.showwarning("Warning", "Enter file name!")
        return
    result = create_file(name)
    status_label.config(text=result)
    refresh_files()


def rename():
    old = file_entry.get()
    new = new_entry.get()
    if old == "" or new == "":
        messagebox.showwarning("Warning", "Enter both file names!")
        return
    result = rename_file(old, new)
    status_label.config(text=result)
    refresh_files()


def delete_file():
    selected = file_list.get(tk.ACTIVE)
```

```
    if selected == "":
        messagebox.showwarning("Warning", "Select a file to delete!")
        return

    confirm = messagebox.askyesno("Confirm Delete",
                                  f"Are you sure you want to delete
'{selected}'?")

    if confirm:
        try:
            os.remove(selected)
            status_label.config(text="File deleted successfully!")
            refresh_files()
        except:
            messagebox.showerror("Error", "Unable to delete file!")

def refresh_files():
    file_list.delete(0, tk.END)
    files = os.listdir()
    for file in files:
        file_list.insert(tk.END, file)

def clear_fields():
    file_entry.delete(0, tk.END)
```

```
    new_entry.delete(0, tk.END)


# ---------- WINDOW SETUP ----------


window = tk.Tk()

window.title("Advanced GUI File Manager")

window.geometry("650x500")

window.configure(bg="#f4f6f9")


# ---------- TITLE ----------


title = tk.Label(window, text="□ FILE MANAGER SYSTEM",

                 font=("Arial", 20, "bold"),

                 bg="#2c3e50", fg="white", pady=12)

title.pack(fill="x")


# ---------- INPUT FRAME ----------


input_frame = tk.Frame(window, bg="#f4f6f9", pady=20)

input_frame.pack()


tk.Label(input_frame, text="File Name:", bg="#f4f6f9",

         font=("Arial", 12)).grid(row=0, column=0, padx=10, pady=5)
```

```
file_entry = tk.Entry(input_frame, width=35)
file_entry.grid(row=0, column=1, pady=5)

tk.Label(input_frame, text="New Name:", bg="#f4f6f9",
         font=("Arial", 12)).grid(row=1, column=0, padx=10, pady=5)

new_entry = tk.Entry(input_frame, width=35)
new_entry.grid(row=1, column=1, pady=5)

# ---------- BUTTON FRAME ----------

button_frame = tk.Frame(window, bg="#f4f6f9")
button_frame.pack()

tk.Button(button_frame, text="Create File", width=15,
          bg="#27ae60", fg="white",
          command=create).grid(row=0, column=0, padx=10, pady=10)

tk.Button(button_frame, text="Rename File", width=15,
          bg="#2980b9", fg="white",
          command=rename).grid(row=0, column=1, padx=10, pady=10)
```

```
tk.Button(button_frame, text="Delete File", width=15,
          bg="#c0392b", fg="white",
          command=delete_file).grid(row=0, column=2, padx=10, pady=10)

tk.Button(button_frame, text="Clear Fields", width=15,
          bg="#e67e22", fg="white",
          command=clear_fields).grid(row=0, column=3, padx=10, pady=10)

# ---------- FILE LIST SECTION ----------

list_frame = tk.Frame(window)
list_frame.pack(pady=15)

tk.Label(list_frame, text="□ Files in Current Folder",
         font=("Arial", 14)).pack()

file_list = tk.Listbox(list_frame, width=70, height=10)
file_list.pack()

refresh_files()

# ---------- STATUS BAR ----------
```

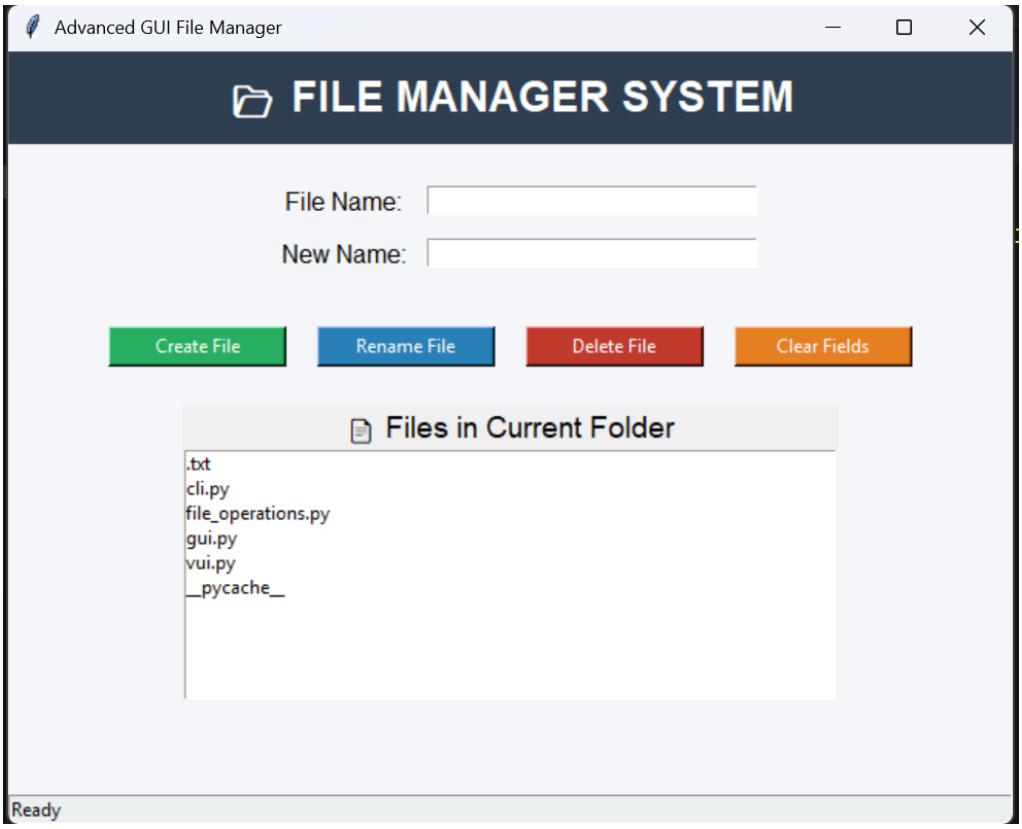
```
status_label = tk.Label(window, text="Ready",
                        bd=1, relief="sunken",
                        anchor="w", bg="#ecf0f1")
status_label.pack(fill="x", side="bottom")


window.mainloop()
```

HOW IT WORKS:

1. The Tkinter library is used to create the graphical interface, and the os module is used for file handling operations.
2. Functions for creating and renaming files are imported from the file_operations module.
3. The main window is created with a title, fixed size, and styled background.
4. Input fields are provided to enter the file name and new file name.
5. Buttons are created for Create, Rename, Delete, and Clear operations.
6. When a button is clicked, the corresponding function executes the required file operation.
7. The Listbox displays all files present in the current directory using the os.listdir() function.
8. A status bar at the bottom displays messages about successful operations or errors.
9. Warning and confirmation message boxes are used to prevent invalid inputs and accidental deletions.

OUTPUT:



3.VUI IMPLEMENTATION(VOICE USER INTERFACE):

THEORY:

A Voice User Interface (VUI) enables users to interact with a system through voice commands instead of keyboard or mouse input.

CODE:

```
import speech_recognition as sr

import pyttsx3

from file_operations import create_file, rename_file



engine = pyttsx3.init()
```

```python
def speak(text):
    engine.say(text)
    engine.runAndWait()


def listen():
    r = sr.Recognizer()
    with sr.Microphone() as source:
        speak("Speak now")
        audio = r.listen(source)
    try:
        return r.recognize_google(audio).lower()
    except:
        return ""


speak("Say create file or rename file")
command = listen()


if "create" in command:
    speak("Say file name")
    filename = listen()
    result = create_file(filename + ".txt")
    speak(result)
```

```python
elif "rename" in command:

    speak("Say old file name")

    old = listen()

    speak("Say new file name")

    new = listen()

    result = rename_file(old + ".txt", new + ".txt")

    speak(result)



else:

    speak("Command not recognized")
```

HOW IT WORKS:

1. The SpeechRecognition library is used to capture voice input through the microphone.
2. The pyttsx3 library is used to provide voice output responses to the user.
3. The program first speaks a prompt asking the user to say a command such as "create file" or "rename file".
4. The `listen()` function records audio and converts it into text using Google Speech Recognition.
5. If the command contains the word "create", the program asks for the file name and creates a new text file.
6. If the command contains the word "rename", it asks for the old and new file names and renames the file.
7. The respective file operation is performed using functions imported from the file_operations module.
8. The result of the operation is spoken back to the user.
9. If the command is not recognized, the system informs the user accordingly.

RESULT:

The CLI, GUI, and VUI based File Manager systems were successfully implemented and tested. The program correctly performed file operations such as creating, renaming, and deleting files.